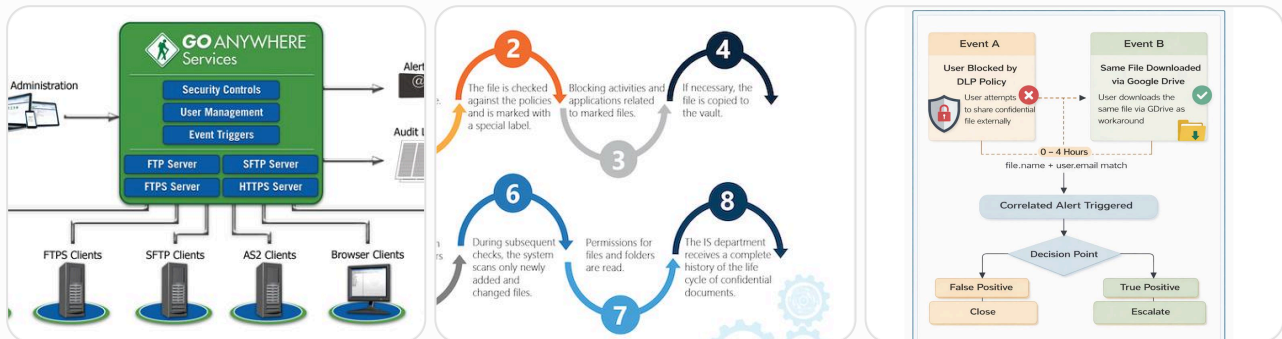
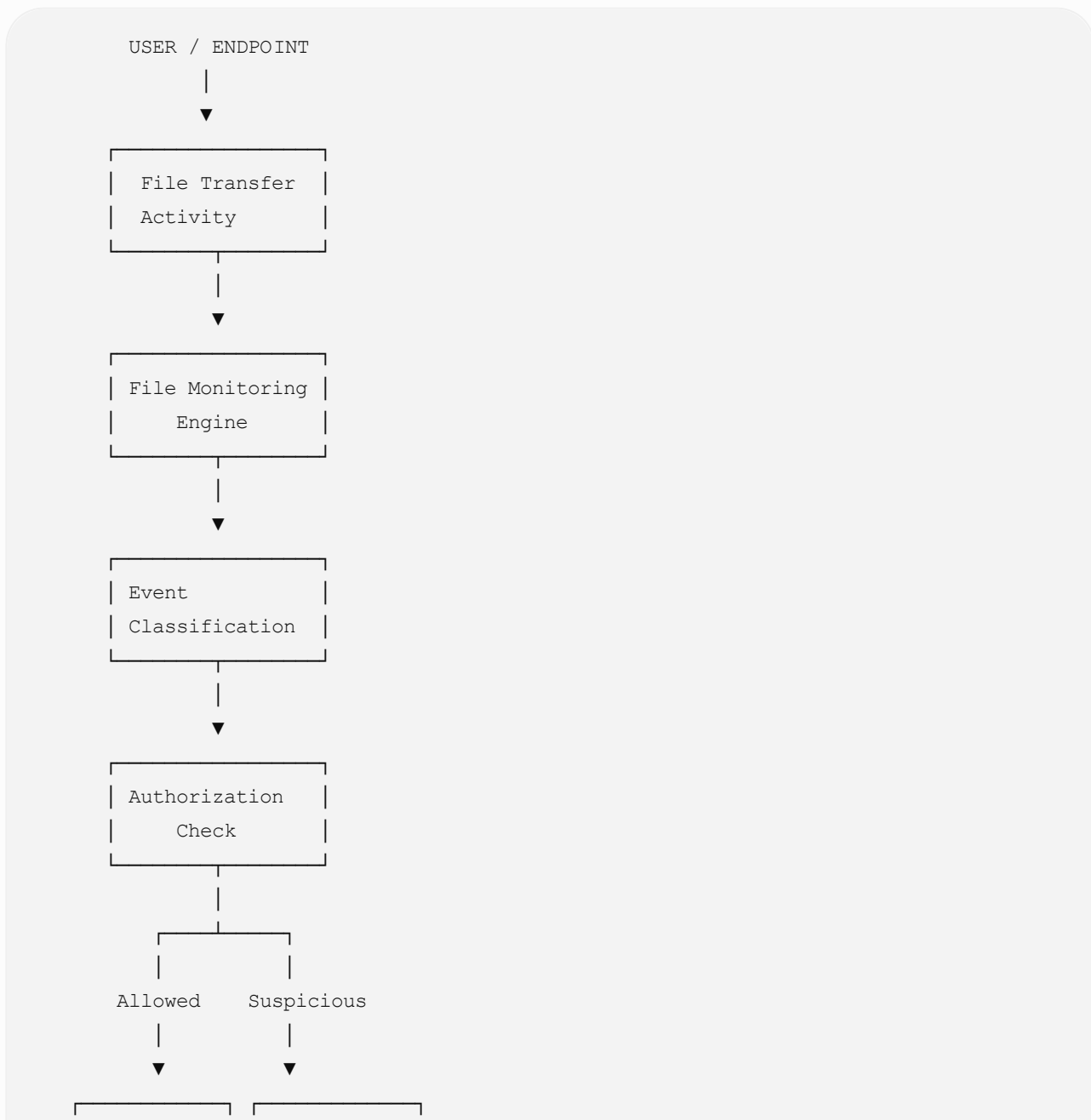


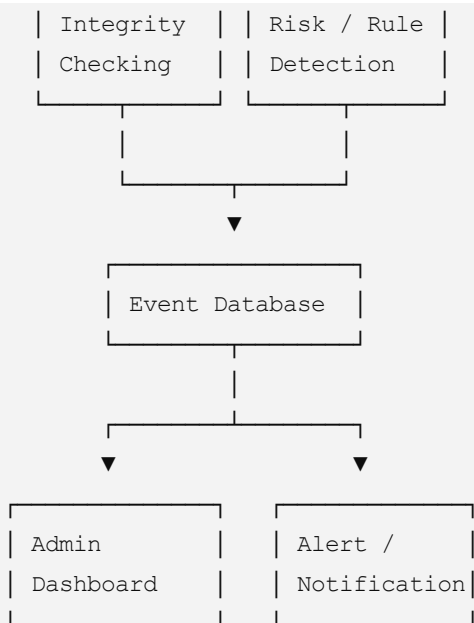
Secure File Transfer Monitoring System — Workflow Architecture

The **Secure File Transfer Monitoring System (SFTMS)** monitors file movement, checks whether transfers are authorized, verifies file integrity, detects suspicious activity, records security events, and alerts the administrator.



1. Overall Workflow





Step 1: User / Endpoint Initiates File Transfer

The workflow starts when a user or endpoint performs a file operation.

Examples:

- Copy a file
- Move a file
- Upload a file
- Download a file
- Modify a file
- Delete a file
- Transfer a file to another directory/system

For example:

User → Copies confidential.pdf → External/Restricted Folder

The system should monitor the relevant folders, endpoints, or transfer services.

Step 2: File Monitoring Engine

The **File Monitoring Engine** detects file-system events.

For a Python implementation, the `Watchdog` library can be used to monitor events such as:

CREATE
MODIFY
MOVE
DELETE

Example:

File: confidential.pdf
Operation: MOVE
Source: C:\Documents
Destination: D:\External
Time: 10:32:15
User: user01

The monitoring engine sends this information to the next stage.

Step 3: Event Collection

The system creates a structured event from the detected activity.

A typical event can contain:

Field	Example
Event ID	EVT-1001
Username	user01
File Name	confidential.pdf
Source	C:\Documents
Destination	D:\External
Operation	MOVE
File Size	2 MB
Timestamp	10:32:15
IP Address	192.168.1.20

This information becomes the basis for security analysis.

Step 4: File Classification

The system determines whether the file is **normal or sensitive**.

Possible classifications:

```
NORMAL
CONFIDENTIAL
SENSITIVE
RESTRICTED
```

Classification can be based on:

- File extension
- Filename
- Location
- Metadata
- Configured rules
- Content classification

Example:

```
resume.pdf      → NORMAL
company_data.xlsx → SENSITIVE
passwords.txt   → RESTRICTED
malware.exe     → HIGH RISK
```

Step 5: Authorization Check

The system checks whether the user is allowed to perform the operation.

For example:

```
User: employee01
File: confidential.pdf
Destination: External Drive

Permission:
Employee → External transfer NOT allowed
```

Therefore:

```
Authorization = DENIED
```

Another example:

```
User: security_admin
File: security_report.pdf
Destination: Approved Server

Authorization = ALLOWED
```

This stage prevents unauthorized file movement.

Step 6: Integrity Verification

For important files, the system calculates a cryptographic hash.

A common algorithm is **SHA-256**.

```
Original File
  |
  ▼
SHA-256
  |
  ▼
Hash A
```

After transfer:

```
Transferred File
  |
  ▼
SHA-256
  |
  ▼
Hash B
```

The system compares:

```
Hash A == Hash B
  |
  ├── YES → File integrity maintained
  |
  └── NO  → Possible modification/corruption
```

This helps detect whether the file changed during the monitored transfer.

Step 7: Security Rule Evaluation

The monitoring system evaluates the event against predefined security rules.

Example rules:

Rule	Condition	Risk
R-01	Sensitive file → restricted destination	High
R-02	Unauthorized user transfer	High
R-03	Executable file transfer	Medium/High
R-04	Excessive file transfers	Medium
R-05	Transfer outside allowed hours	Medium
R-06	SHA-256 mismatch	High

Example:

```
User transfers 50 confidential files
      ↓
Rule R-04 triggered
      ↓
Risk = HIGH
      ↓
Alert generated
```

Step 8: Risk Scoring

The system can assign a risk level to every event.

```
LOW
MEDIUM
HIGH
CRITICAL
```

For example:

```
Normal PDF
      ↓
Authorized
      ↓
LOW RISK
```

Whereas:

```
Sensitive file
+
Unauthorized user
+
External destination
↓
CRITICAL RISK
```

A numerical scoring system can also be used:

```
0-30      → Low
31-60     → Medium
61-80     → High
81-100    → Critical
```

Step 9: Event Logging

Every important activity is stored in the database.

Example:

```
Event ID: EVT-1005
User: user01
File: confidential.xlsx
Operation: COPY
Source: C:\Company
Destination: E:\USB
Authorization: DENIED
Risk: HIGH
Hash: SHA256...
Time: 10:45:22
```

A database such as **SQLite** can be used for a student/lab project, while PostgreSQL can be considered for larger deployments.

Step 10: Alert Generation

If suspicious activity is detected, the system generates an alert.

Example:

⚠ SECURITY ALERT

Unauthorized File Transfer Detected

User: user01

File: confidential.xlsx

Destination: External Drive

Risk: HIGH

Time: 10:45:22

Possible notification methods include:

- Dashboard alert
- Email
- Webhook
- SIEM notification
- Log-based alert

Step 11: Administrator Dashboard

The administrator can view the collected information through a Flask-based web dashboard.

Example dashboard:

SECURE FILE TRANSFER MONITOR				
Total Events	Allowed	Blocked	Alerts	
1250	1100	150	42	
Recent Security Events				
user01	confidential.pdf	HIGH	BLOCKED	
user02	report.pdf	LOW	ALLOWED	
user03	malware.exe	HIGH	ALERT	

The dashboard can provide:

- Event statistics
- File transfer logs
- Suspicious activity
- User activity
- Risk levels
- Search and filtering

- Alert management
 - Reports
-

Step 12: Incident Response

When a high-risk event is detected, the administrator investigates it.

Example:

```
Alert
↓
Review User
↓
Review File
↓
Review Destination
↓
Check Authorization
↓
Check Hash
↓
Investigate Activity
↓
Take Action
```

Possible actions:

```
Allow
Block
Quarantine
Disable account
Preserve evidence
Investigate further
```

Complete End-to-End Example

Suppose an employee attempts to copy a confidential company file to an unauthorized USB drive.

Step 1

```
Employee copies payroll.xlsx
```

Step 2

Monitoring engine detects:

```
Operation = COPY
```

Step 3

System collects:

```
User = employee01  
File = payroll.xlsx  
Destination = USB
```

Step 4

File classification:

```
payroll.xlsx → SENSITIVE
```

Step 5

Authorization:

```
USB transfer → NOT PERMITTED
```

Step 6

Detection rule:

```
Sensitive file + unauthorized destination
```

Step 7

Risk:

```
HIGH
```

Step 8

Event is stored in the database.

Step 9

Alert is generated:

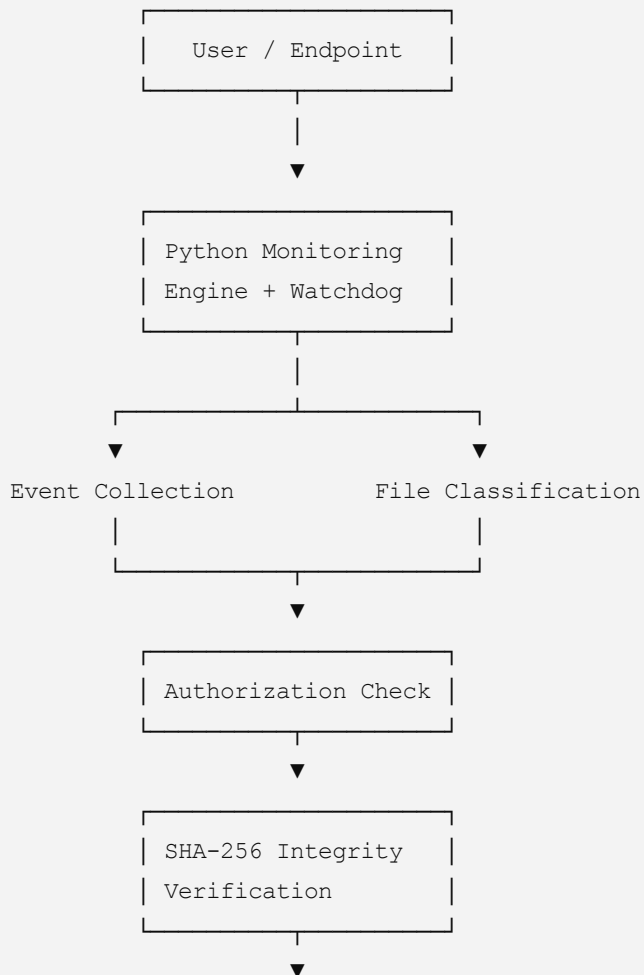
Step 10

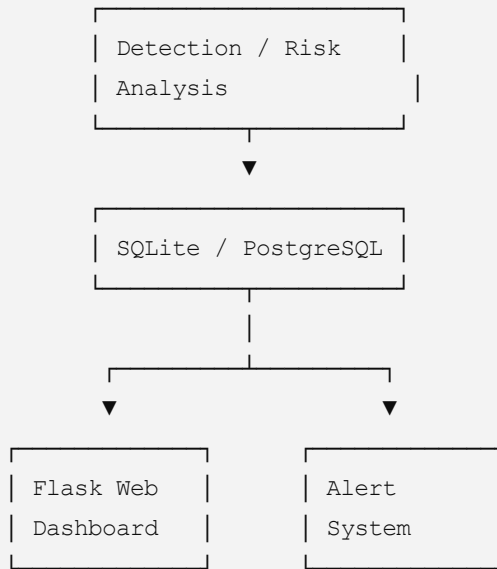
Administrator sees the event on the dashboard.

```
Employee01
  ↓
payroll.xlsx
  ↓
USB
  ↓
Unauthorized
  ↓
HIGH RISK
  ↓
ALERT
```

Technology Architecture

For your project, a practical implementation can use:





Final Workflow

Monitor → Collect → Classify → Authorize → Hash → Detect → Risk Score → Log → Alert → Dashboard → Investigate

This workflow is suitable for your **cybersecurity internship project** because it demonstrates practical concepts in **security monitoring, file integrity, access control, event logging, threat detection, alerting, and incident response**.

Files, images, and data analysis are unavailable until usage resets at 12:44 PM. Continue chatting with text only, or upgrade for more access.

[Try Plus free](#)